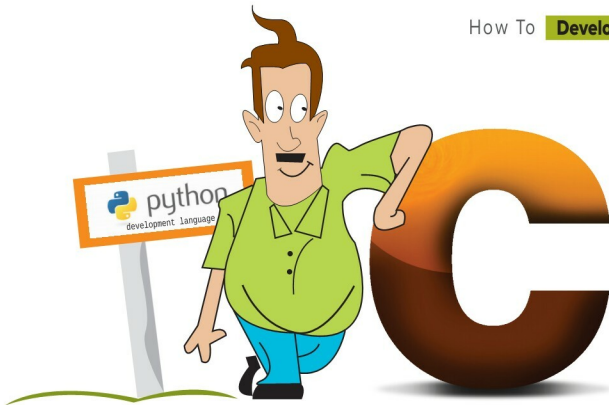




When Python Weds C

It is a well-known fact that the C language is closely related to many programming languages. This article discusses the Application Programming Interfaces (APIs) between Python and C. The Python/C APIs are widely used to extend the Python interpreter using C (what we'll briefly explore here) or to embed Python inside a C application. The references given at the end of the article can be used to delve deeper into the topic.

This article assumes that:

- You have a basic understanding of C and Python.
- The required development environment is already in place.

Python version x.y (I used 2.6 for our discussion) is installed under the `PYTHON_LIB_PATH` directory (e.g. `/usr/lib/`), and Python headers are under `PYTHON_INCLUDE_PATH` (e.g. `/usr/include`).

Sample module source

Let's begin by writing a C module with the functions to print 'Hello world', to add integers and to decode an example structure. The module is later used from a Python program. The following is the source code (with line numbering); the file, named `python_interface.c`, is available on the website at <http://bit.ly/fJZfb9>.

```
1 #include <Python.h>
2
3 static PyObject* InterfaceMethod_hello_world(PyObject *self)
4 {
5     printf("Hello World!\n");
6     Py_INCREF(Py_None);
7     return Py_None;
```

```
8 }
9
10 static PyObject* InterfaceMethod_add_integers(PyObject *self,
11 PyObject *args)
12 {
13     int a=0, b=0, c=0, sum=0;
14     PyObject *retsum;
15
16     if(PyArg_ParseTuple(args, "iii", &a, &b, &c) == 0) {
17         printf("Error in parsing\n");
18         PyErr_SetString(PyExc_StandardError, "Error in parsing");
19         return NULL;
20     }
21     sum=a+b+c;
22     printf("sum = %d\n", sum);
23     retsum=Py_BuildValue("i", sum);
24     return retsum;
25 }
26 #pragma pack(1)
27 typedef union {
28     unsigned int val;
29     struct header {
```